

Multi-Agent Legal Explainer

A LangGraph-orchestrated agent over the Egyptian Civil Code

Technical Report • Stack: Claude Agent SDK • LangGraph • LightRAG knowledge graph • Ollama Qwen3 embeddings • NLTK Punkt • Gradio UI

1. Overview & Objective

The system answers natural-language legal questions about the Egyptian Civil Code by orchestrating **three specialised Claude subagents** behind a **LangGraph** state machine. An LLM-based *router* classifies each query as *simple*, *medium*, or *complex* and dispatches it down the cheapest path that yields a correct, grounded answer. Tools replace recall with lookup: a curated bilingual glossary, a deterministic article-keyed statute index, a graph-RAG retriever, and an optional web search. Every claim is cited; the system refuses rather than hallucinates.

The package implements the two product epics targeted in this work:

- **Epic 6 — Tooling & Function Calling.** Strict-schema tools registered as a single MCP server, so the model reaches for a deterministic function instead of guessing definitions or article text.
- **Epic 7 — Multi-Agent Collaboration.** A baseline two-role pipeline (*Researcher* → *Explainer*) is shipped alongside the orchestrated flow, and a parallel eval harness compares the two on the same 424-case RAFT validation set.

2. System Architecture

The orchestrator is a LangGraph state machine. Each node is an async function that mutates a shared `GraphState`; conditional edges route between nodes based on the router’s decision. Subagents are themselves wrapped `query()` calls to Claude, each with its own system prompt and tool allow-list — they are the *nodes* of the graph, not orchestrators.

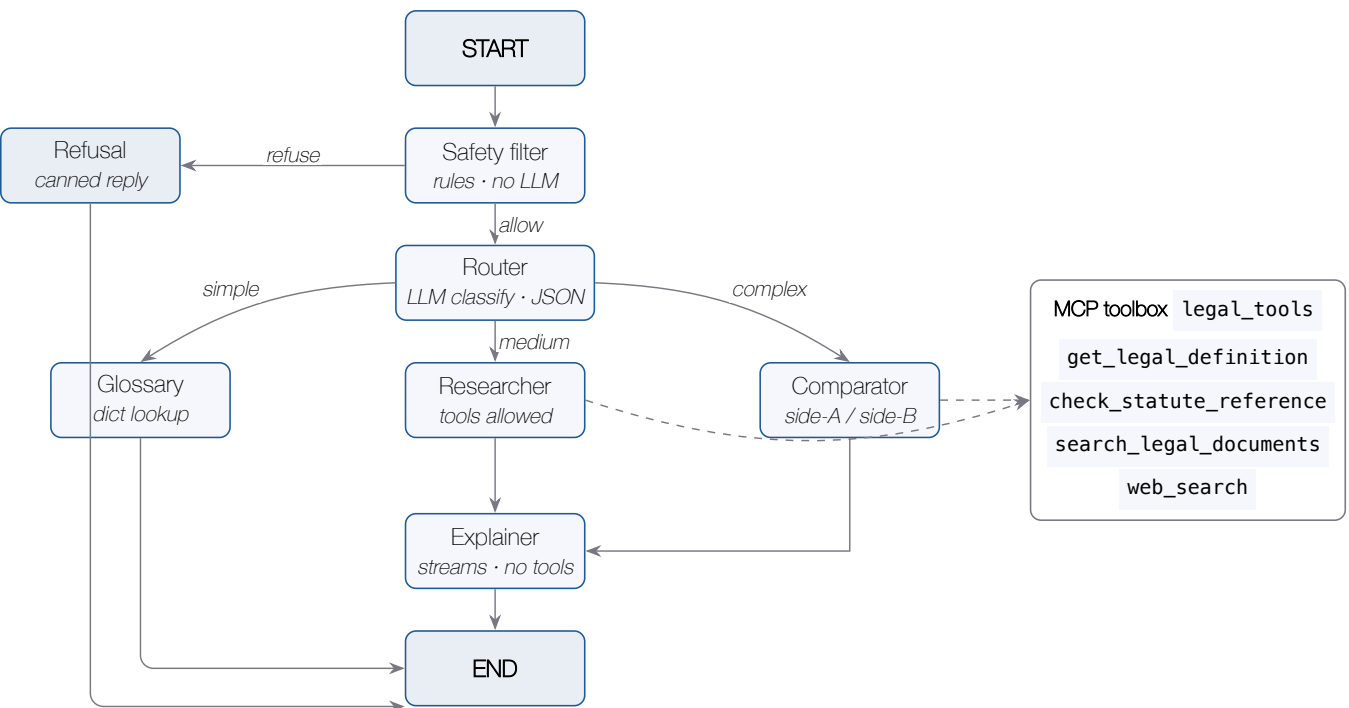


Figure 1 – Compiled LangGraph orchestrator. Bold boxes are terminal nodes (START, END, refusal); rounded boxes are agent or tool nodes. Dashed arrows show MCP tool access — only Researcher and Comparator may call tools; Explainer never does (it grounds strictly on the JSON it receives from the previous node).

3. Subagents & Models

Role	Model	Why this choice
Orchestrator (LangGraph state machine)	deterministic Python	No LLM in the control plane; pure flow logic + edge predicates compiled once.
Router	Claude (single classify call)	One short prompt → JSON {complexity, glossary_term_id, reason}; defensively validated against the glossary so a hallucinated id down-grades to medium.
Researcher (T≈0.2)	Claude with MCP tools	Retrieves and extracts; emits structured JSON (key_passages , key_facts , ambiguities). Never synthesizes.
Comparator (T≈0.3)	Claude with MCP tools	Two-sided structured output (side_a , side_b , shared_ground , differences); prompt forces side-isolated retrieval.
Explainer (T≈0.4)	Claude, no tools	Receives structured findings, writes the user-facing answer; mandatory disclaimer; language matches the user query. Streams chunks for live UI.

The three subagents share the same MCP toolbox but with different allow-lists. The Explainer is intentionally toolless: its only grounding is the JSON the previous subagent produced, which structurally prevents it from inventing facts.

4. Methodology — Tool Catalog (Epic 6)

Tools are bundled into one MCP server (legal_tools) via the Claude Agent SDK's create_sdk_mcp_server ; each subagent registers it and exposes only the subset it is permitted to call. Tool descriptions tell the model when to call and when not to, keeping the catalog small and high-signal.

Tool	What it does	Backing store
get_legal_definition	Bilingual EN/AR glossary of 15 curated civil-law terms with cross-language aliases (indemnity, compensation, and the Arabic equivalents all resolve to one canonical entry).	data/glossary.json
check_statute_reference	Parses citations in Article N or Art. N format (English and Arabic accepted) and returns the verbatim article text. Returns {found: false} on miss rather than fabricating.	articles_index.json (2,164 articles, parsed once from EgyptianLaw.pdf)
search_legal_documents	Graph-RAG retrieval over the Egyptian Civil Code knowledge graph. Returns structured entities, relationships, and source chunks; never invokes its own LLM for synthesis or keyword extraction (see §6).	LightRAG aquery_data()
web_search	DuckDuckGo wrapper for current-events / recent-amendment questions outside the static corpus.	ddgs package

Failure mode discipline. Every tool returns a structured {found: false, reason: ...} or {ok: false, message: ...} on a miss, never an exception. The model is instructed to honour these and surface them to the user, which is the difference between an honest “I don’t have Article 1341” and a confidently wrong answer.

5. Methodology — Orchestration (Epic 7)

5.1 The LLM Router

A single Claude classification call replaces the brittle keyword regex of v1. The router prompt embeds the 15 canonical glossary ids inline so the model knows what simple actually means, and asks for strict JSON:

```
{"complexity": "simple|medium|complex",
 "glossary_term_id": "<id or null>",
 "reason": "<one short sentence>"}
```

The Python side defensively verifies that any `glossary_term_id` actually exists in the glossary; otherwise it downgrades to *medium*. This eliminates a class of bugs (an LLM-hallucinated id leading to a nonsense glossary lookup) that occurred in the v1 rule-based router.

5.2 Path Dispatch

Path	When chosen	LLM calls	Typical latency
<code>simple</code>	Router identifies a single glossary term	1 (router)	~1 s
<code>medium</code>	Article reference or single-topic query	3 (router + Researcher + Explainer)	~50 s
<code>complex</code>	Comparison / contrast keywords detected	3 (router + Comparator + Explainer)	~70 s
<code>refused</code>	Safety filter matched off-policy patterns (case-advice, document drafting, out-of-jurisdiction)	0	~0 s

5.3 State & Observability

`GraphState` is a `TypedDict` that each node enriches via partial updates — cost and duration accumulate as nodes return. A `FlowLogger` captures every event (safety verdict, router decision, subagent start/end, tool call, tool result, streaming answer chunks) into both an in-memory list (for the live UI) and a JSONL trace file at `reports/agent_traces/`. The same logger is read by the Gradio app to render the live “thinking” panel and stream the answer token-by-token.

6. Knowledge Base (brief)

Retrieval is secondary to this report, but the agent’s grounding depends on it, so a one-paragraph summary: `EgyptianLaw.pdf` is parsed once with PyMuPDF, then `LightRAG` chunks (~4,000-token chunks with 200-token overlap), embeds each chunk with `Ollama Qwen3-Embedding 0.6B` (1024-dim, multilingual), and builds an entity/relation knowledge graph backed by NanoVectorDB. The result: 2,164 articles indexed, 1,285 entities, 1,474 relations. A separate deterministic pass produces `articles_index.json` (article-keyed verbatim text) used by `check_statute_reference` for citation-grade lookups.

No-LLM retrieval invariant. The `search_legal_documents` tool calls `LightRAG’s aquery_data()` with pre-extracted `hl_keywords` and `ll_keywords` computed by NLTK Punkt + bilingual stopwords + glossary/article-ref matching, which makes `LightRAG` bypass its internal LLM-based extractor entirely. A defensive `_llm_guard` callback raises if any code path tries to invoke an LLM inside the retrieval — so we cannot silently burn tokens during tool use.

7. Evaluation

The eval harness runs the orchestrator (or the baseline two-role pipeline) against the existing repo’s RAFT validation set (`qa_pairs_raft_val.jsonl`, 424 bilingual cases). It is asyncio-parallel by default with crash-safe streaming JSONL output, and computes one metric absent from the original judge harness: `retrieval_hit@k` — does the prediction cite the dataset’s gold `article_key`?

Stage	Technique / output
Generation	<code>run_orchestrated_lg(query)</code> or <code>run_baseline(query)</code> per case; parallel via <code>asyncio.Semaphore(N)</code>
Retrieval correctness	Regex on the gold <code>article_key</code> — <i>Article N</i> (or its Arabic equivalent) present in the prediction
Quality scoring	Delegated to the repo’s existing Claude-judge harness (<code>scripts/closed_book_recall_eval.py</code> , 5-dim rubric: legal accuracy, completeness, house style, language, faithfulness)
Latency / cost	Per-case <code>duration_ms</code> and <code>total_cost_usd</code> from each <code>OrchestratorResult</code> ; p50/p90 in summary
Path attribution	Tracks which orchestrator path served each case (simple/medium/complex/refused) for routing-effectiveness analysis

A `snapshot.py` script reads the streaming `.jsonl.partial` file mid-run, so a long sweep can be monitored live without

disrupting it. The companion `compare_baseline.py` prints a side-by-side latency / cost / hit-rate / path-distribution table between two prediction files.

8. Interface — Gradio App

A Gradio UI (`gradio_app.py`) lets a user type a question and observe the agent in real time. Three live panes update as the run progresses:

- **Answer** — the Explainer's final answer streams chunk-by-chunk (the SDK's `TextBlock` s feed a callback that appends to `FlowLogger.answer_text` ; the UI polls every 400 ms). A `|` sentinel renders as a cursor while streaming.
- **Thinking** — a collapsible accordion shows every flow event as it happens: query, safety verdict, router decision (including the LLM's reasoning), subagent start / end with cost and duration, every tool call with its arguments, every tool result with a one-line summary.
- **Run metrics** — live aggregate: elapsed time, current path, tools called, tool call count, subagents active.

Pressing Enter in the prompt textbox submits the query. Six pre-built example queries demonstrate each path (simple glossary, medium English, medium Arabic, complex comparison, safety refusal). The interface is intentionally theme-aware via Gradio's CSS variables so it renders correctly in both light and dark mode.

9. Key Implementation Notes

- **Two engines side-by-side.** A legacy rule-based orchestrator (v1, `orchestrator.py`) is preserved for regression; the LangGraph + LLM-router orchestrator (v2, `orchestrator_langgraph.py`) is the default. The eval runner picks via `--engine {langgraph,legacy}` , and both produce predictions in an identical JSON shape so the Claude-judge harness scores them unchanged.
- **No mid-orchestration LLM in the retrieval path.** LightRAG normally calls an LLM for query-time keyword extraction; we pre-compute `hl_keywords` and `ll_keywords` deterministically and pass them on `QueryParam` , then guard with a callback that errors loudly if LightRAG still tries. This makes the tool's cost predictable.
- **Structured failure beats fabrication.** Every tool returns `{found: false, reason: ...}` for misses. The system prompts tell each subagent to honour these — so a question about an article outside the corpus produces an honest “I don't have Article 1341” instead of an invented one.
- **Bilingual by design.** The glossary, the statute-citation parser, the keyword extractor, the safety patterns, the system prompts, and the disclaimer template are all dual-language; the Explainer's prompt mandates “respond in the user's language.”
- **Crash-safe eval.** The parallel runner appends every completed prediction to a `.jsonl.partial` file before the final `.json` is written, so a 7-hour run that crashes at case 220/424 still has 220 usable predictions on disk.

End-to-end at a glance: `query` → `safety` → `LLM router` → `{glossary | Researcher | Comparator}` → `Explainer (streams)` → `cited answer + disclaimer`